

如何在 Cloud Run GPU 執行 TorchServe 和 Stable Diffusion

來源：<https://codelabs.developers.google.com/codelabs/how-to-use-stable-diffusion-cloud-run-gpu?hl=zh-tw>

步驟數：8

瞭解如何在 Cloud Run GPU 上執行穩定的擴散作業

目錄

1. [簡介](#)
2. [啟用 API 並設定環境變數](#)
3. [建立 Torchserve 應用程式](#)
4. [設定 Cloud NAT](#)
5. [建構及部署 Cloud Run 服務](#)
6. [測試服務](#)
7. [恭喜！](#)
8. [清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

Cloud Run 最近新增了 GPU 支援功能。目前僅開放公開測試，且須先加入候補名單。如有興趣試用這項功能，請[填寫這份表單](#)，加入等候名單。Cloud Run 是 Google Cloud 的容器平台，可讓您輕鬆在容器中執行程式碼，不必管理叢集。

我們目前提供的 GPU 是 Nvidia L4 GPU，具有 24 GB 的 vRAM。每個 Cloud Run 執行個體都有一個 GPU，且 Cloud Run 自動調整資源配置功能仍適用。包括最多擴充至 5 個執行個體 (可申請提高配額)，以及在沒有任何要求時縮減至零個執行個體。

在本程式碼研究室中，您將建立及部署 TorchServe 應用程式，該應用程式會使用 [stable diffusion XL](#)，根據文字提示詞生成圖片。系統會以 Base64 編碼字串的形式，將生成的圖片傳回給呼叫者。

這個範例是以「[Running Stable diffusion model using Huggingface Diffusers in Torchserve](#)」為基礎。本程式碼研究室會說明如何修改這個範例，以便搭配 Cloud Run 使用。

課程內容

- 如何在 Cloud Run 上使用 GPU 執行 Stable Diffusion XL 模型
-

步驟 2 · 約 0 分鐘

2. 啟用 API 並設定環境變數

開始進行本程式碼研究室之前，請先啟用數個 API。本程式碼研究室需要使用下列 API。執行下列指令即可啟用這些 API：

```
gcloud services enable run.googleapis.com \
  storage.googleapis.com \
  cloudbuild.googleapis.com
```

接著，您可以設定本程式碼研究室全程都會用到的環境變數。

```
PROJECT_ID=<YOUR_PROJECT_ID>
REPOSITORY=<YOUR_REPOSITORY_ID>

NETWORK_NAME=default
REGION=us-central1
IMAGE=us-central1-docker.pkg.dev/$PROJECT_ID/$REPOSITORY/gpu-torchserve
```

您為 `REPOSITORY` 設定的值，是儲存映像檔建構作業的 Artifact Registry 存放區。您可以使用現有帳戶或建立新帳戶：



```
gcloud artifacts repositories create $REPOSITORY \
  --location=us-central1 \
  --repository-format=docker
```

步驟 3 · 約 0 分鐘

3. 建立 Torchserve 應用程式

首先，請建立原始碼的目錄，然後 cd 到該目錄。



```
mkdir stable-diffusion-codelab && cd $_
```

建立 `config.properties` 檔案。這是 TorchServe 的設定檔。

```
inference_address=http://0.0.0.0:8080
enable_envvars_config=true
min_workers=1
max_workers=1
default_workers_per_model=1
default_response_timeout=1000
load_models=all
max_response_size=655350000
# to enable authorization, see
https://github.com/pytorch/serve/blob/master/docs/token_authorization_api.md#how-to-set-and-disable-token-authorization
disable_token_authorization=true
```

注意：這個範例不會使用授權來存取 TorchService。如要啟用驗證功能，請參閱 https://github.com/pytorch/serve/blob/master/docs/token_authorization_api.md#how-to-set-and-disable-token-authorization

請注意，在本範例中，[監聽位址](http://0.0.0.0) `http://0.0.0.0` 用於在 Cloud Run 上運作。Cloud Run 的預設通訊埠為 8080。

建立 `requirements.txt` 檔案。

```
python-dotenv
accelerate
transformers
diffusers
numpy
google-cloud-storage
nvgpu
```

建立名為 `stable_diffusion_handler.py` 的檔案

```

from abc import ABC
import base64
import datetime
import io
import logging
import os

from diffusers import StableDiffusionXLImg2ImgPipeline
from diffusers import StableDiffusionXLPipeline
from google.cloud import storage
import numpy as np
from PIL import Image
import torch
from ts.torch_handler.base_handler import BaseHandler

logger = logging.getLogger(__name__)

def image_to_base64(image: Image.Image) -> str:
    """Convert a PIL image to a base64 string."""
    buffer = io.BytesIO()
    image.save(buffer, format="JPEG")
    image_str = base64.b64encode(buffer.getvalue()).decode("utf-8")
    return image_str

class DiffusersHandler(BaseHandler, ABC):
    """Diffusers handler class for text to image generation."""

    def __init__(self):
        self.initialized = False

    def initialize(self, ctx):
        """In this initialize function, the Stable Diffusion model is loaded and
        initialized here.

        Args:
            ctx (context): It is a JSON Object containing information pertaining to
                the model artifacts parameters.
        """
        logger.info("Initialize DiffusersHandler")
        self.manifest = ctx.manifest
        properties = ctx.system_properties
        model_dir = properties.get("model_dir")
        model_name = os.environ["MODEL_NAME"]
        model_refiner = os.environ["MODEL_REFINER"]

```

```

self.bucket = None

logger.info(
    "GPU device count: %s",
    torch.cuda.device_count(),
)
logger.info(
    "select the GPU device, cuda is available: %s",
    torch.cuda.is_available(),
)
self.device = torch.device(
    "cuda:" + str(properties.get("gpu_id"))
    if torch.cuda.is_available() and properties.get("gpu_id") is not None
    else "cpu"
)
logger.info("Device used: %s", self.device)

# open the pipeline to the inferenece model
# this is generating the image
logger.info("Downloading model %s", model_name)
self.pipeline = StableDiffusionXLPipeline.from_pretrained(
    model_name,
    variant="fp16",
    torch_dtype=torch.float16,
    use_safetensors=True,
).to(self.device)
logger.info("done downloading model %s", model_name)

# open the pipeline to the refiner
# refiner is used to remove artifacts from the image
logger.info("Downloading refiner %s", model_refiner)
self.refiner = StableDiffusionXLImg2ImgPipeline.from_pretrained(
    model_refiner,
    variant="fp16",
    torch_dtype=torch.float16,
    use_safetensors=True,
).to(self.device)
logger.info("done downloading refiner %s", model_refiner)

self.n_steps = 40
self.high_noise_frac = 0.8
self.initialized = True
# Commonly used basic negative prompts.
logger.info("using negative_prompt")
self.negative_prompt = ("worst quality, normal quality, low quality, low res,
blurry")

# this handles the user request

```

```

def preprocess(self, requests):
    """Basic text preprocessing, of the user's prompt.

    Args:
        requests (str): The Input data in the form of text is passed on to the
            preprocess function.

    Returns:
        list : The preprocess function returns a list of prompts.
    """
    logger.info("Process request started")
    inputs = []
    for _, data in enumerate(requests):
        input_text = data.get("data")
        if input_text is None:
            input_text = data.get("body")
        if isinstance(input_text, (bytes, bytearray)):
            input_text = input_text.decode("utf-8")
        logger.info("Received text: '%s'", input_text)
        inputs.append(input_text)
    return inputs

def inference(self, inputs):
    """Generates the image relevant to the received text.

    Args:
        input_batch (list): List of Text from the pre-process function is passed
            here

    Returns:
        list : It returns a list of the generate images for the input text
    """
    logger.info("Inference request started")
    # Handling inference for sequence_classification.
    image = self.pipeline(
        prompt=inputs,
        negative_prompt=self.negative_prompt,
        num_inference_steps=self.n_steps,
        denoising_end=self.high_noise_frac,
        output_type="latent",
    ).images
    logger.info("Done model")

    image = self.refiner(
        prompt=inputs,
        negative_prompt=self.negative_prompt,
        num_inference_steps=self.n_steps,
        denoising_start=self.high_noise_frac,

```



```

        image=image,
    ).images
    logger.info("Done refiner")

    return image

def postprocess(self, inference_output):
    """Post Process Function converts the generated image into Torchserve readable
    format.

    Args:
        inference_output (list): It contains the generated image of the input
            text.

    Returns:
        (list): Returns a list of the images.
    """
    logger.info("Post process request started")
    images = []
    response_size = 0
    for image in inference_output:
        # Save image to GCS
        if self.bucket:
            image.save("temp.jpg")

            # Create a blob object
            blob = self.bucket.blob(
                datetime.datetime.now().strftime("%Y%m%d_%H%M%S") + ".jpg"
            )

            # Upload the file
            blob.upload_from_filename("temp.jpg")

            # to see the image, encode to base64
            encoded = image_to_base64(image)
            response_size += len(encoded)
            images.append(encoded)

    logger.info("Images %d, response size: %d", len(images), response_size)
    return images

```

建立名為 `start.sh` 的檔案。這個檔案會做為容器中的進入點，啟動 TorchServe。

```
#!/bin/bash

echo "starting the server"
# start the server. By default torchserve runs in backaround, and start.sh will
immediately terminate when done
# so use --foreground to keep torchserve running in foreground while start.sh is
running in a container
torchserve --start --ts-config config.properties --models
"stable_diffusion=${MAR_FILE_NAME}.mar" --model-store ${MAR_STORE_PATH} --foreground
```

然後執行下列指令，將其設為可執行檔。

```
chmod 755 start.sh
```

建立 `dockerfile` 。

```
# pick a version of torchserve to avoid any future breaking changes
# docker pull pytorch/torchserve:0.11.1-cpp-dev-gpu
FROM pytorch/torchserve:0.11.1-cpp-dev-gpu AS base

USER root

WORKDIR /home/model-server

COPY requirements.txt ./
RUN pip install --upgrade -r ./requirements.txt

# Stage 1 build the serving container.
FROM base AS serve-gcs

ENV MODEL_NAME='stabilityai/stable-diffusion-xl-base-1.0'
ENV MODEL_REFINER='stabilityai/stable-diffusion-xl-refiner-1.0'

ENV MAR_STORE_PATH='/home/model-server/model-store'
ENV MAR_FILE_NAME='model'
RUN mkdir -p $MAR_STORE_PATH

COPY config.properties ./
COPY stable_diffusion_handler.py ./
COPY start.sh ./

# creates the mar file used by torchserve
RUN torch-model-archiver --force --model-name ${MAR_FILE_NAME} --version 1.0 --
handler stable_diffusion_handler.py -r requirements.txt --export-path
${MAR_STORE_PATH}

# entrypoint
CMD [ "./start.sh" ]
```

步驟 4 · 約 0 分鐘

4. 設定 Cloud NAT

Cloud NAT 可提供更高的頻寬，讓您存取網際網路並從 HuggingFace 下載模型，大幅縮短部署時間。

注意：Cloud NAT 不像 Cloud Run 採用即付即用計費模式。請參閱 Cloud NAT 定價說明文件]
(<https://cloud.google.com/nat/pricing>)。

如要使用 Cloud NAT，請執行下列指令來啟用 Cloud NAT 執行個體：

```
gcloud compute routers create nat-router --network $NETWORK_NAME --region us-central1
gcloud compute routers nats create vm-nat --router=nat-router --region=us-central1 --
auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges
```

步驟 5 · 約 0 分鐘

5. 建構及部署 Cloud Run 服務

將程式碼提交至 Cloud Build。

```
gcloud builds submit --tag $IMAGE
```

接著，部署至 Cloud Run

```
gcloud beta run deploy gpu-torchserve \  
  --image=$IMAGE \  
  --cpu=8 --memory=32Gi \  
  --gpu=1 --no-cpu-throttling --gpu-type=nvidia-l4 \  
  --allow-unauthenticated \  
  --region us-central1 \  
  --project $PROJECT_ID \  
  --execution-environment=gen2 \  
  --max-instances 1 \  
  --network $NETWORK_NAME \  
  --vpc-egress all-traffic
```

步驟 6 · 約 0 分鐘

6. 測試服務

您可以執行下列指令來測試服務：

```
PROMPT_TEXT="a cat sitting in a magnolia tree"

SERVICE_URL=$(gcloud run services describe gpu-torchserve --region $REGION --format
'value(status.url)')

time curl $SERVICE_URL/predictions/stable_diffusion -d "data=$PROMPT_TEXT" | base64 -
-decode > image.jpg
```

如果 Cloud Run 設有驗證機制，請務必將授權標頭新增至 curl 指令。

您會在目前的目錄中看到 `image.jpg` 檔案。您可以在 Cloud Shell 編輯器中開啟圖片，查看坐在樹上的貓。

步驟 7 · 約 0 分鐘

7. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱 [Cloud Run GPU](#) 說明文件。

涵蓋內容

- 如何在 Cloud Run 上使用 GPU 執行 Stable Diffusion XL 模型
-

步驟 8 · 約 0 分鐘

8. 清除所用資源

為避免產生意外費用 (例如，如果這個 Cloud Run 工作意外遭到呼叫的次數，超過[免費層級的每月 Cloud Run 呼叫配額](#))，您可以刪除 Cloud Run 工作，或刪除您在步驟 2 中建立的專案。

如要刪除 Cloud Run 工作，請前往 Cloud Run Cloud 控制台 (<https://console.cloud.google.com/run/>) 並刪除 `gpu-torchserve` 服務。

您也需要刪除 [Cloud NAT 設定](#)。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
